



Apuntes de Compiladores e Intérpretes

Clase: 20/08/2026

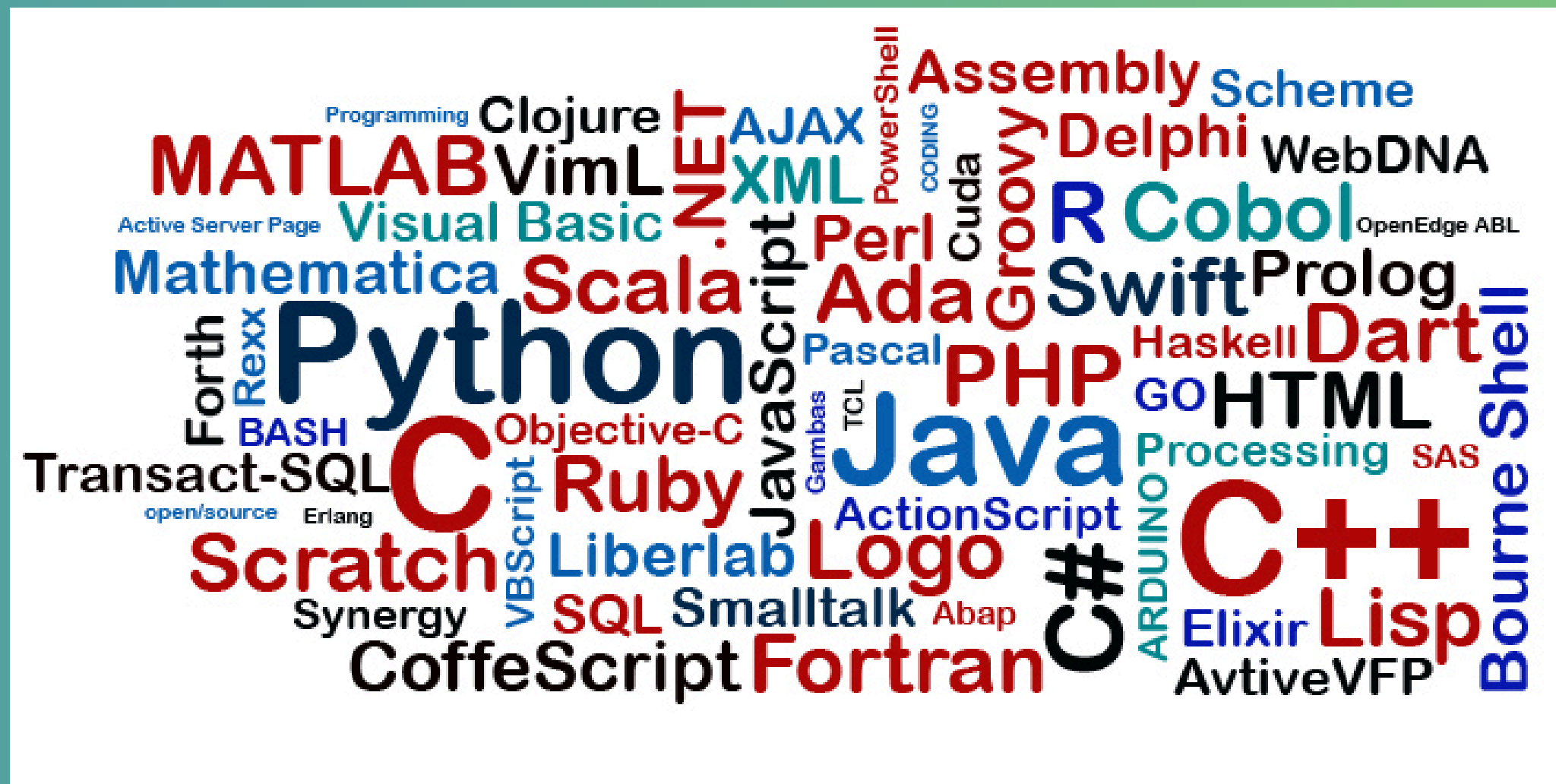
Hecho por: Luis Felipe Benavides Solórzano



Máquinas Virtuales y Traducción

¿Qué es crear una máquina virtual?

Debemos diseñar el lenguaje que la define, y lograr que ese lenguaje sea entendido por otra maquina (virtual o real)

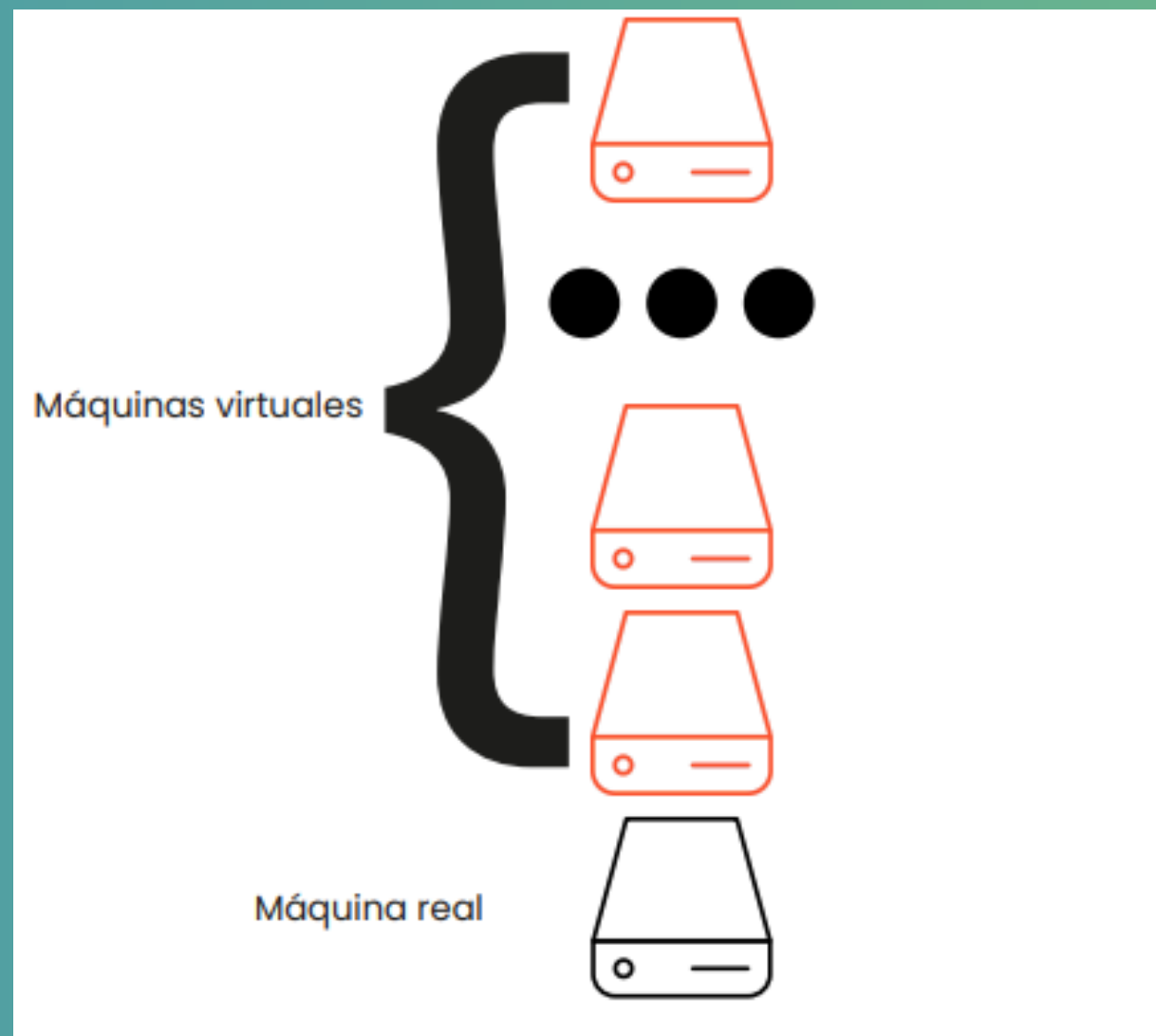


En la práctica suele ser con un lenguaje de alto nivel

Máquinas Virtuales y Traducción

Una torre de maquinas

Cada máquina virtual se construye sobre la anterior. Si llamamos M_i a una máquina, esta se apoya en M_{i-1} , y M_i asume que M_{i-1} es real, aunque en el fondo también sea virtual



Esto genera un problema: hay que mapear el lenguaje L_i de la máquina M_i en el lenguaje L_{i-1} de la máquina anterior

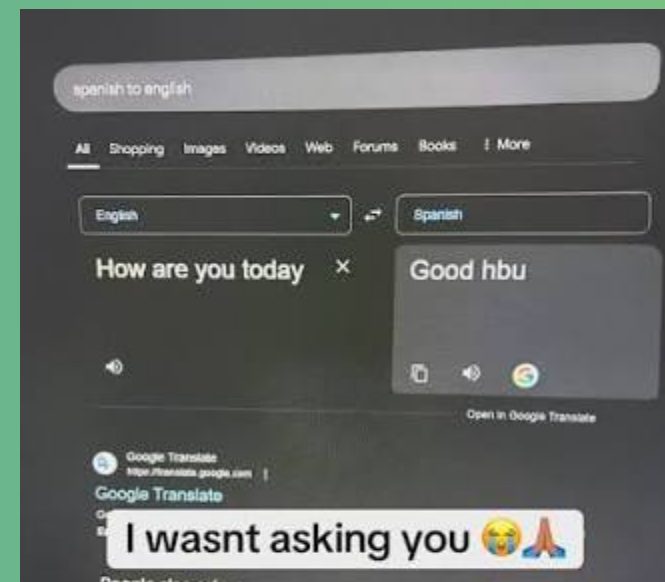
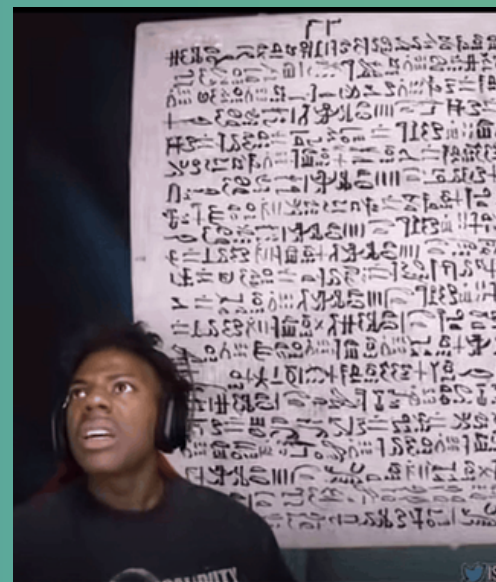


Traducir vs. interpretar

Hay 2 formas de resolver ese mapeo entre lenguajes:

Traducir: Convertir el programa completo escrito en L_i a un programa equivalente en L_{i-1} . Hay que esperar a que termine la traducción antes de poder ejecutar, pero se hace una sola vez y después el programa corre rápido

Interpretar: Tomar cada instrucción que la máquina M_i quiere ejecutar y simular su ejecución directamente, instrucción por instrucción. Es más lento, porque esa simulación se repite cada vez que corre el programa

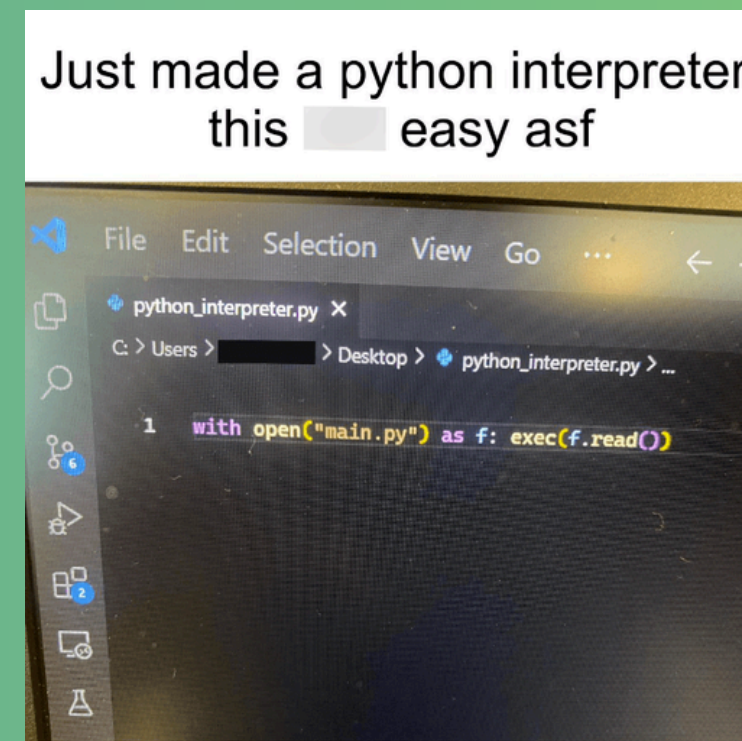


Traducir vs. interpretar

De eso se crea:

Compilador: Software que traduce un programa escrito en el lenguaje de una máquina virtual a un programa equivalente escrito en el lenguaje de otra máquina virtual. El nombre lo eligió Grace Hopper

Intérprete: Software que ejecuta un programa escrito en el lenguaje de una máquina virtual corriendo una simulación de esa máquina sobre otra máquina virtual

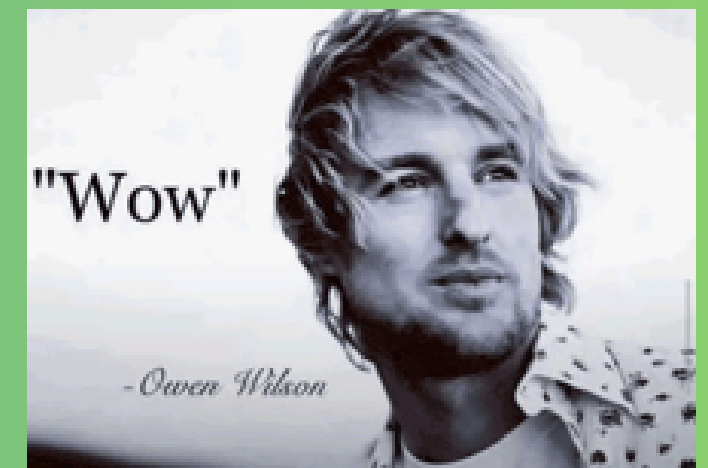
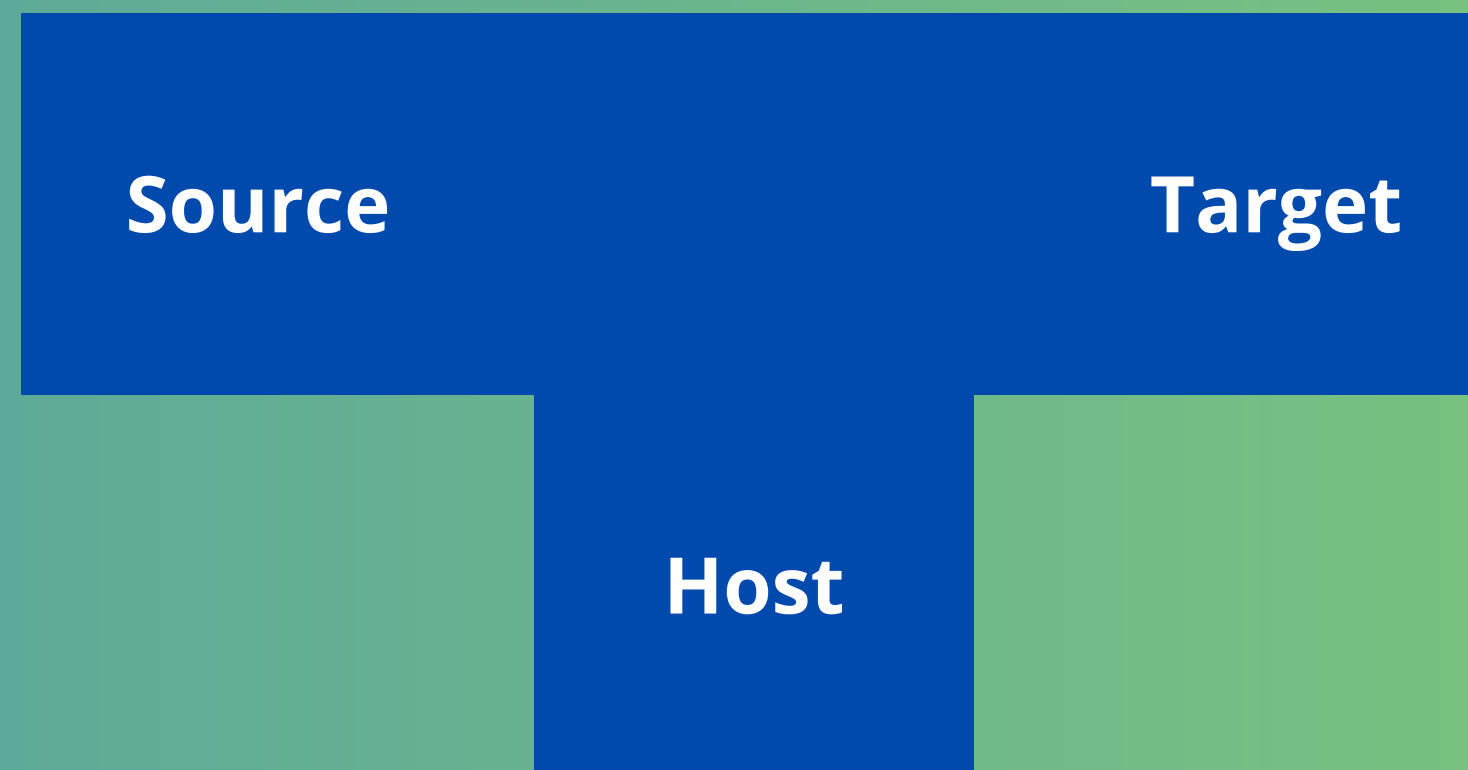


Diagramas T

Es la forma estándar de representar un compilador. Todo compilador tiene tres lenguajes asociados:

- Lenguaje Fuente (Source): La entrada que hay que traducir
- Lenguaje Objeto (Target): La salida hacia la que se traduce
- Lenguaje de Implementación (Host): En el que está escrito el propio compilador

Representación:



Diagramas T

Ejemplo 1:

Leng. C

x86

x86

Chat,
is this real?



Fanta-C = Fantasy
(mal chiste, ríanse
porfa)

Diagramas T

ES REAL!

GCC: GNU C COMPILER, después renombrado a GNU COMPILER COLLECTION

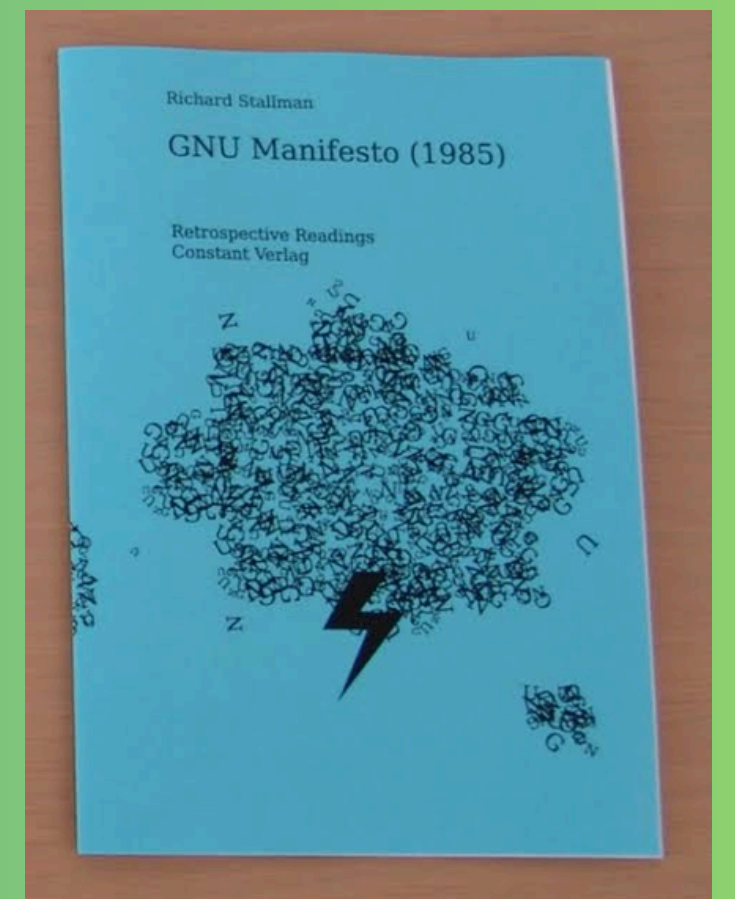
Lo escribió originalmente Richard Stallman (Not Kirstein btw)

Estándar para GNU, Linux, etc

Leng. C

x86

x86



Diagramas T

Ejemplo 2:

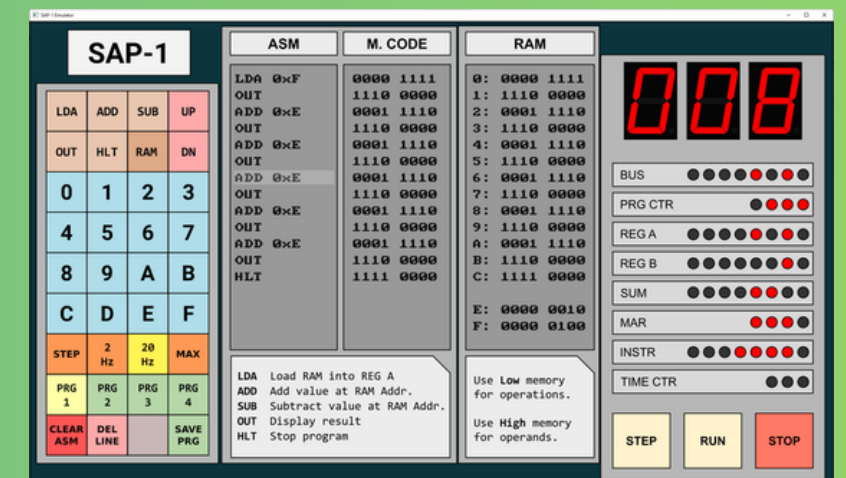
Ens. IBM 704

Leng. Maq.
704

Leng. Maq.
704

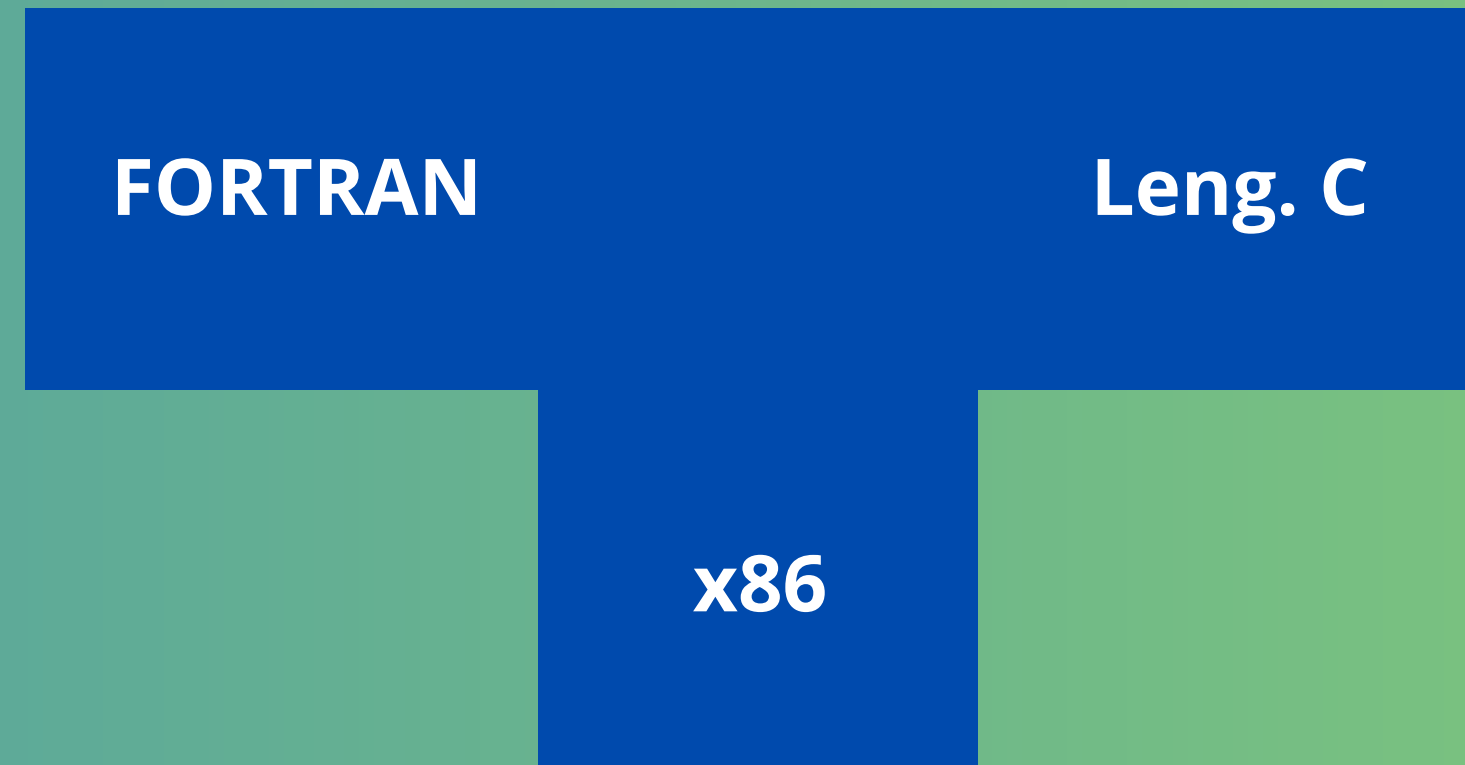
Symbolic Assembly Program (SAP): Ensamblador para la IBM 704

- Escrito por Roy Nutt en lenguaje maquina 704
- Hecho en 1956
- Ya habían intentos previos de ensambladores, pero este fue el mejor
- Influenció todos los ensambladores del futuro
- John Backus escribió FORTRAN en SAP



Diagramas T

Ejemplo 3:



F2C (FORTRAN TO C):

- Genera código C a partir de código FORTRAN
- Claro, si lo convierte... Pero que sea legible es otra cosa:

```
PROGRAM SUMA
  INTEGER I, N, SUM
  SUM = 0
  DO 10 I = 1, 10
    SUM = SUM + I
  10  CONTINUE
  PRINT *, SUM
  END
```



```
/* Main program translated by f2c (version 20260101). */
#include "f2c.h"
static integer c__9 = 9;

int MAIN__(void)
{
    integer s_wsle(cilist *), do_lio(integer *, integer *, char *, ftnlen),
    e_wsle(void);
    static integer i_, sum;
    static cilist io__2;

    sum = 0;
    for (i_ = 1; i_ <= 10; ++i_) {
        sum += i_;
    }

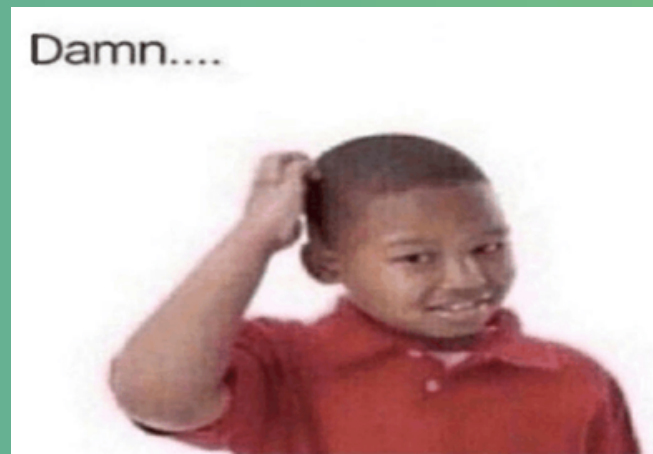
    io__2.ciend = 0;
    io__2.cierr = 0;
    io__2.cifmt = 0;
    s_wsle(&io__2);
    do_lio(&c__9, &i_, (char *)&sum, (ftnlen)sizeof(integer));
    e_wsle();
    return 0;
}
```



Operaciones con Diagramas T

Se pueden combinar estos diagramas para representar procesos de traducción más complejos. Podemos definir un "álgebra de Diagramas T"

Primer caso: Lenguaje Objeto es igual al Lenguaje Fuente

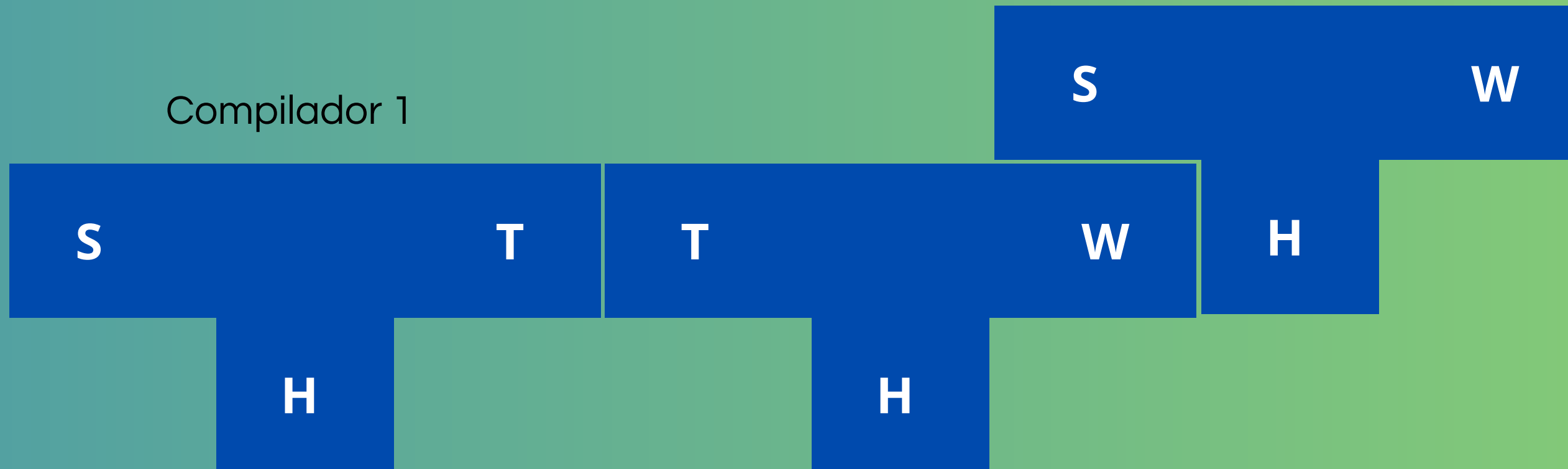


Operaciones con Diagramas T

Notación:

Resultado

Compilador 1

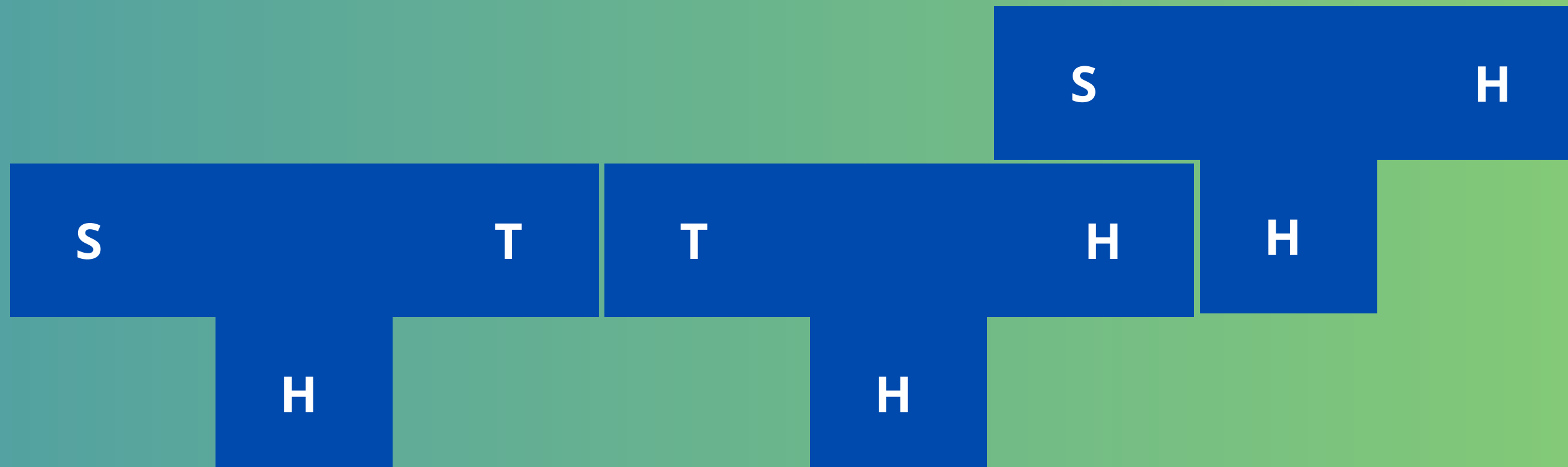
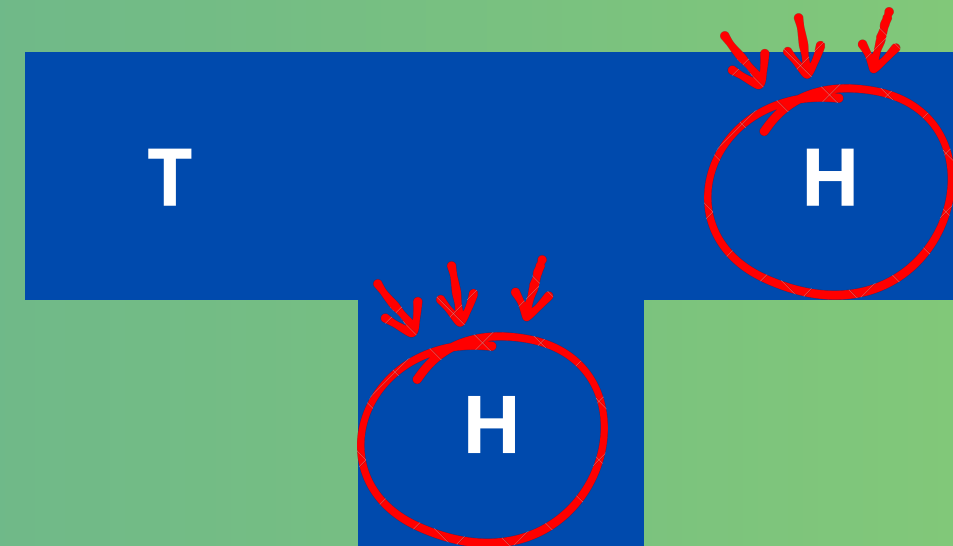
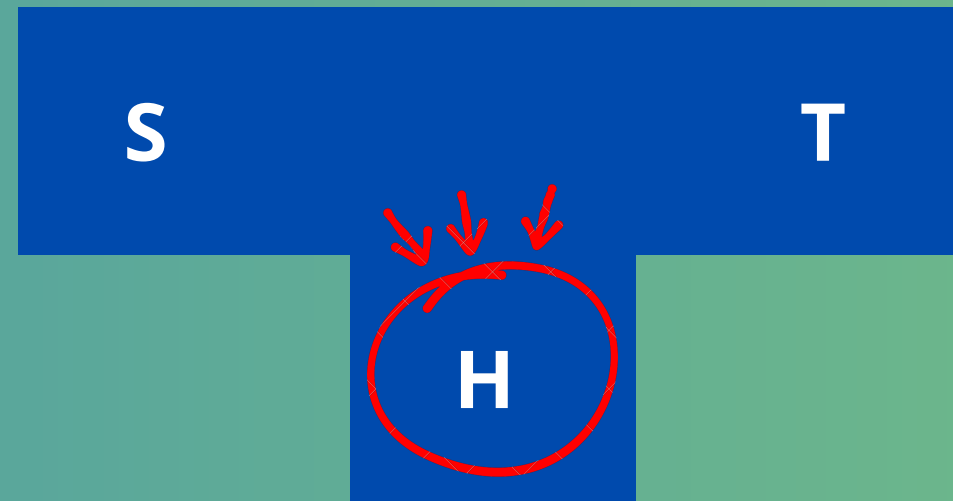


Compilador 2



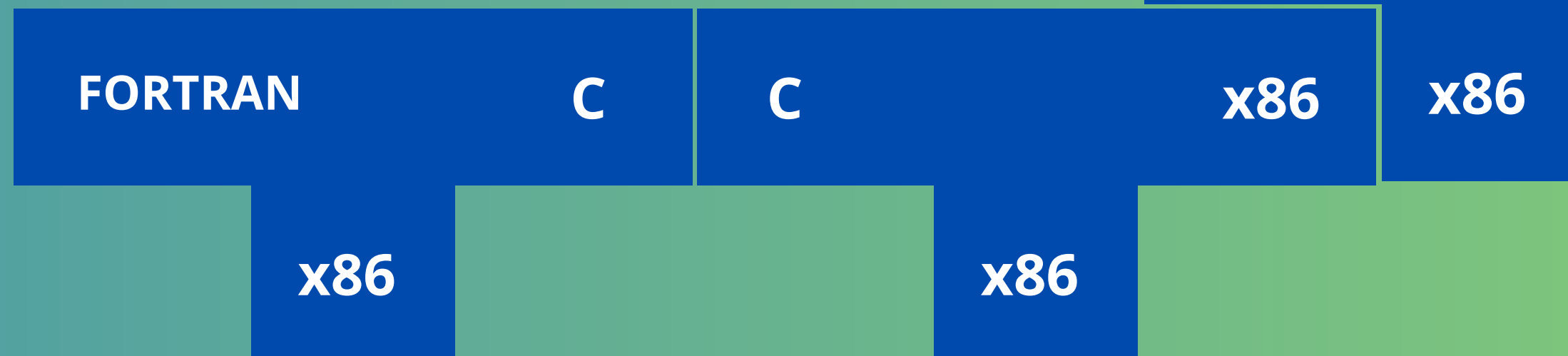
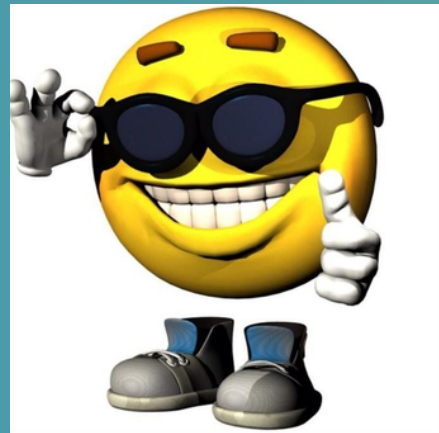
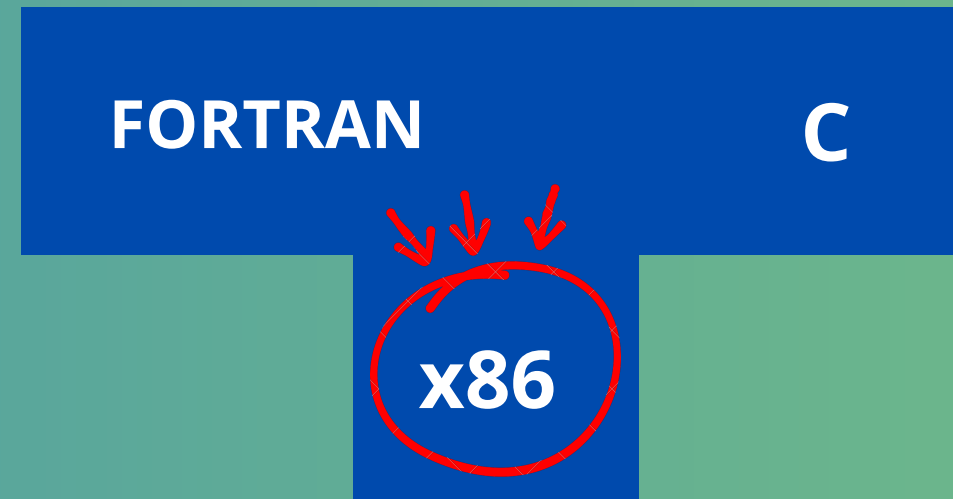
Operaciones con Diagramas T

Caso típico:



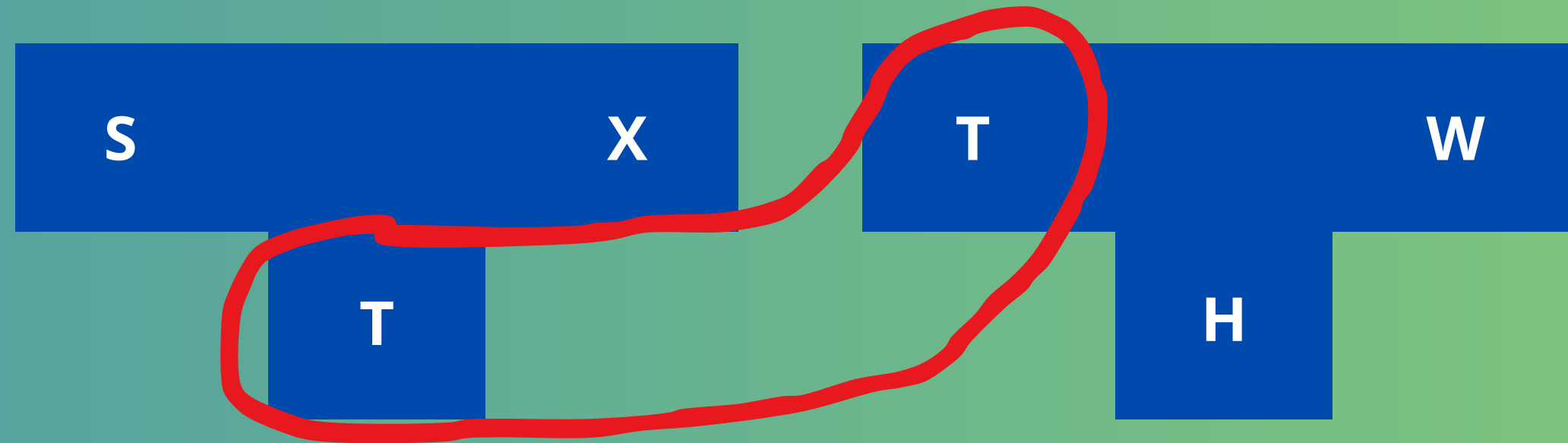
Operaciones con Diagramas T

F2C Y GCC:



Operaciones con Diagramas T

Segundo caso: Lenguaje de Implementación es igual al Lenguaje Fuente

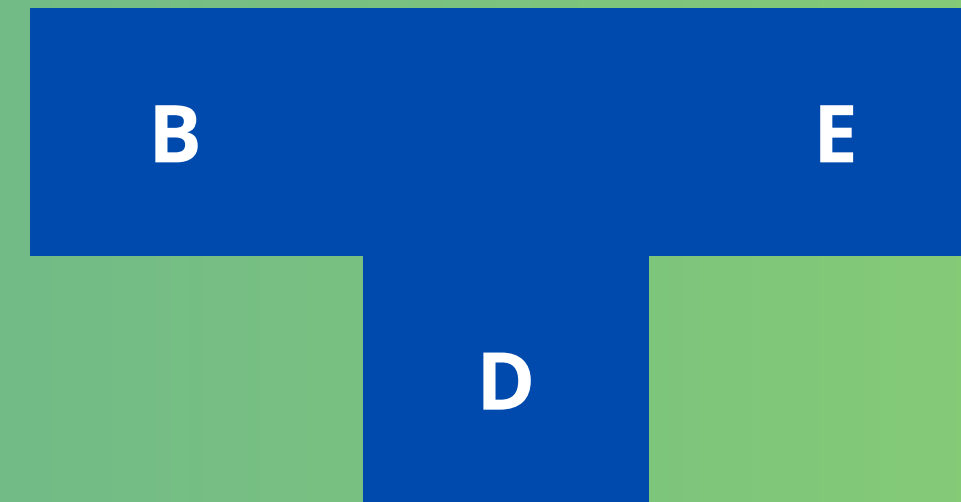
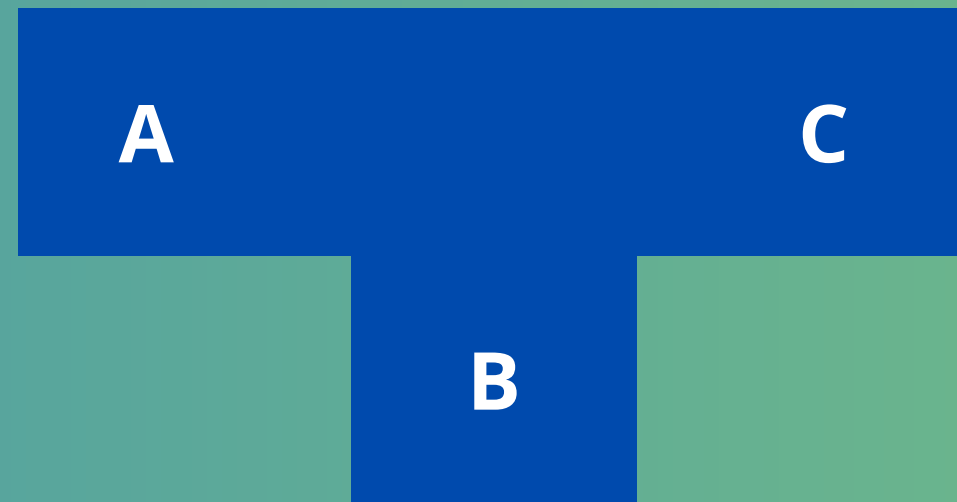


El Lenguaje de Implementación se puede encadenar con otro traductor:

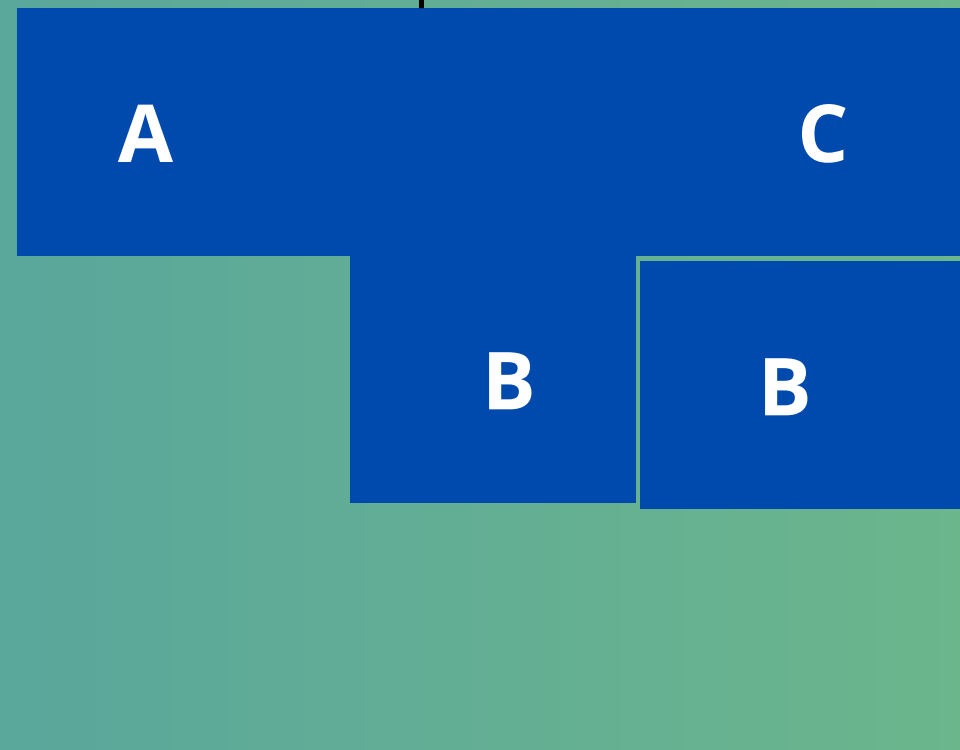


Operaciones con Diagramas T

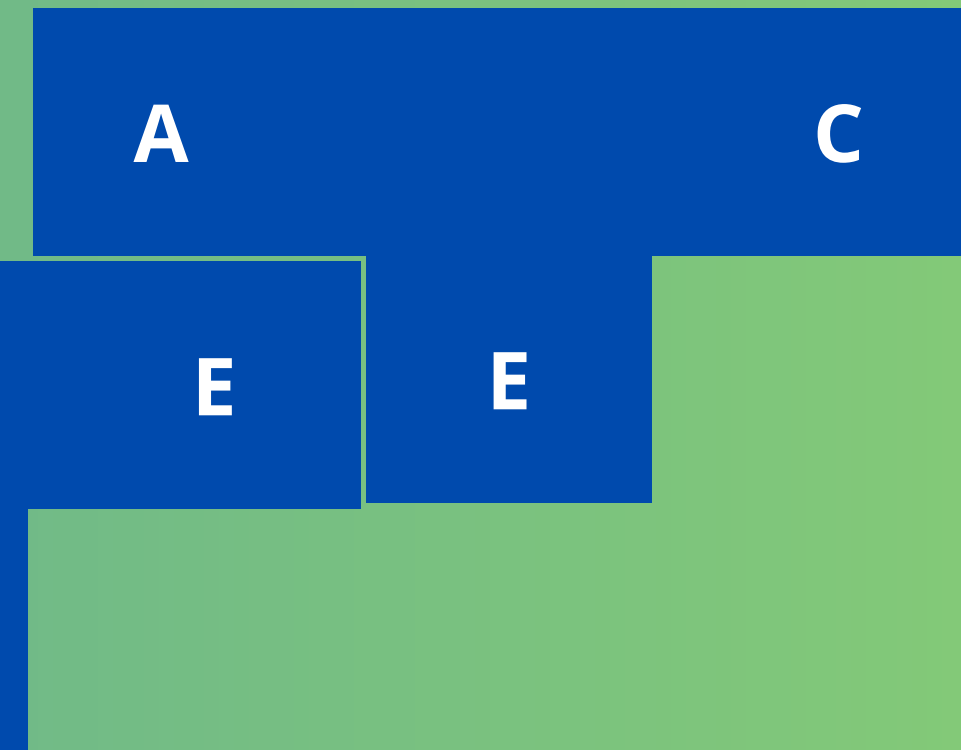
Notación:



Compilador 1



Resultado



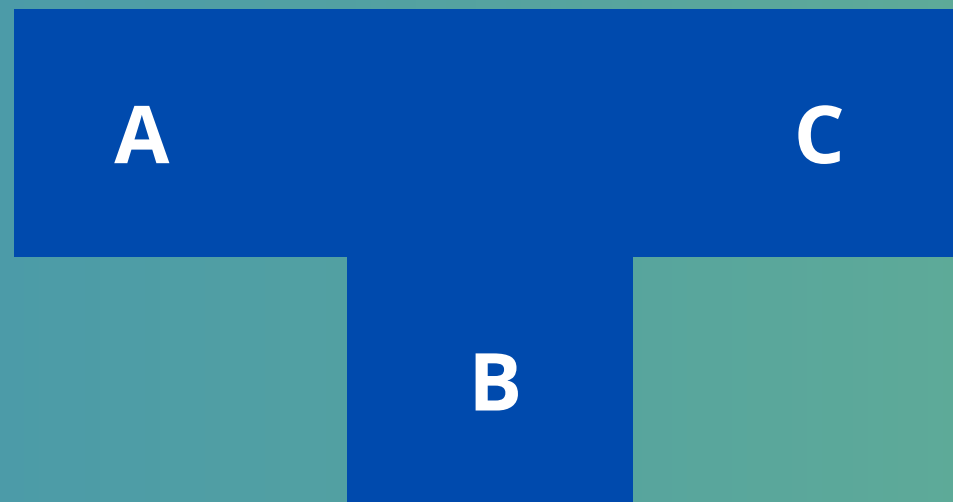
Compilador 2



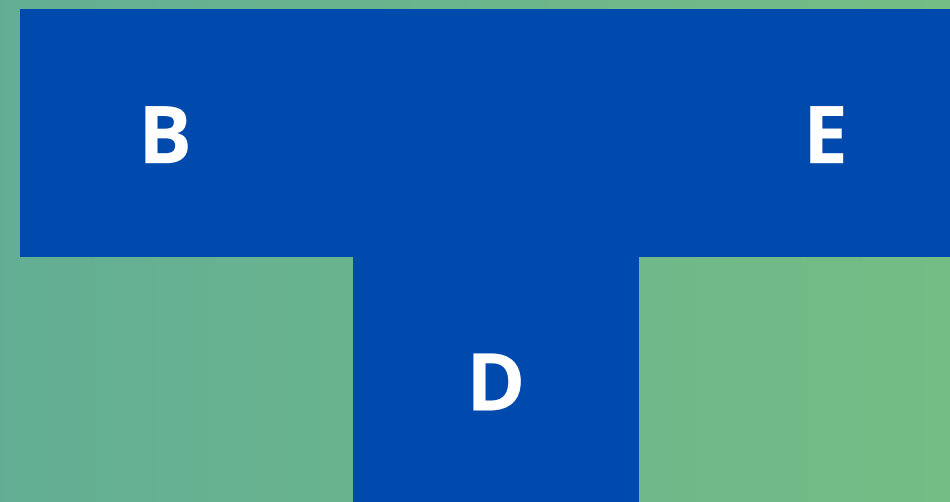
Operaciones con Diagramas T

Pueden ser varios niveles... ¿Cómo resolvemos?

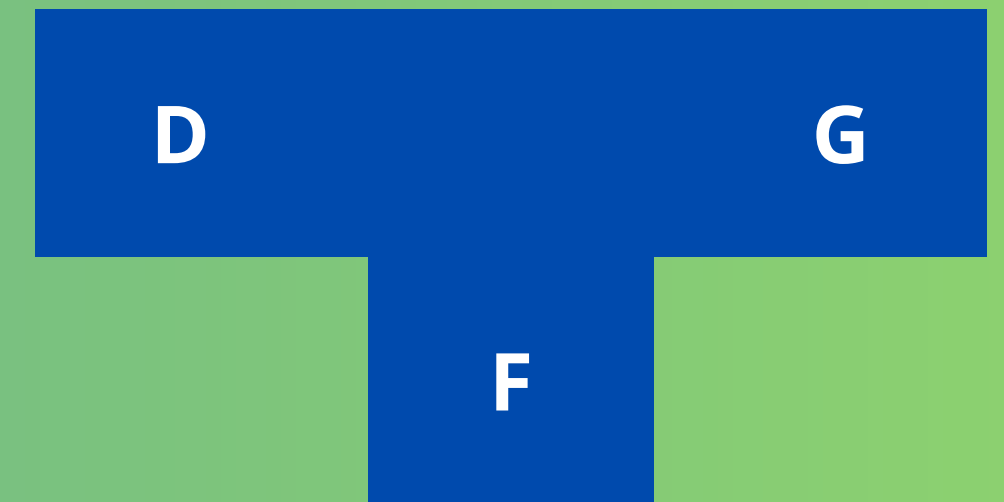
Compilador 1



Compilador 2

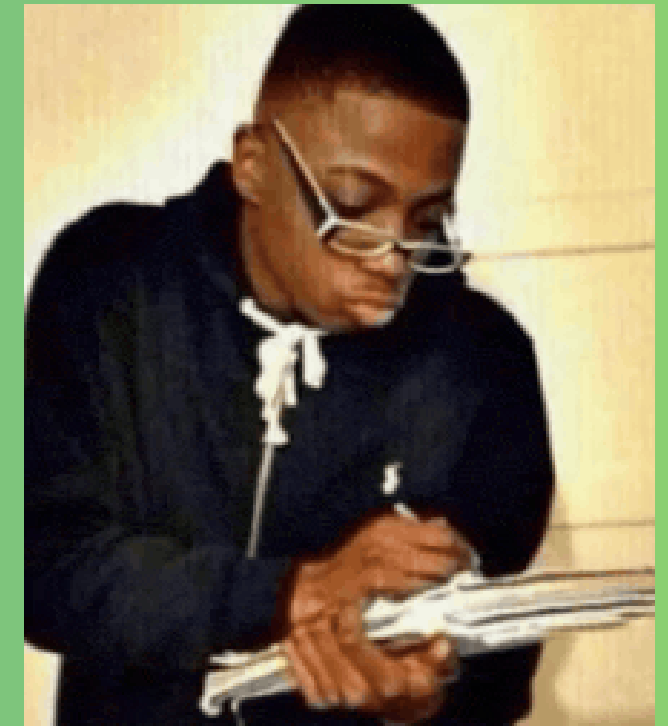
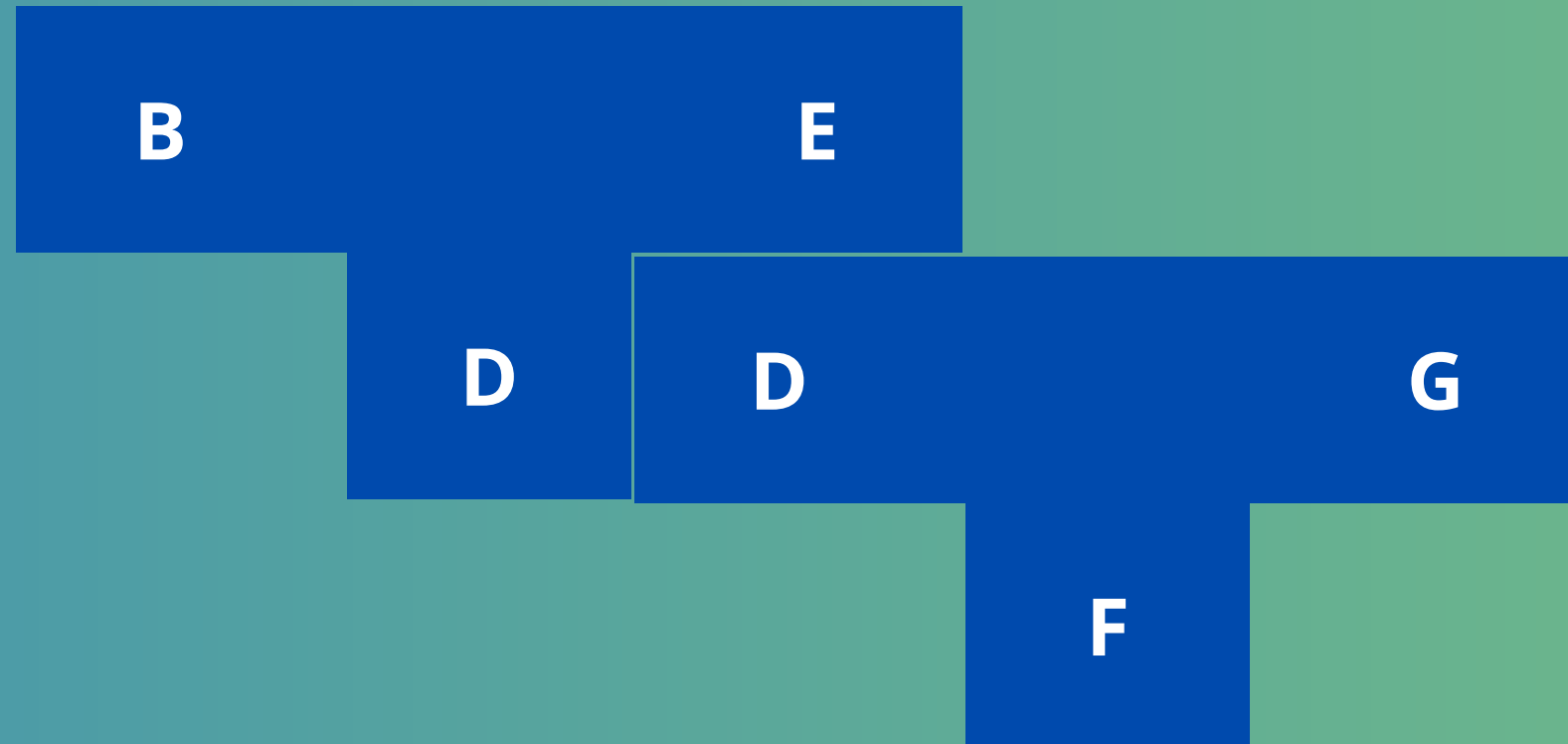


Compilador 3

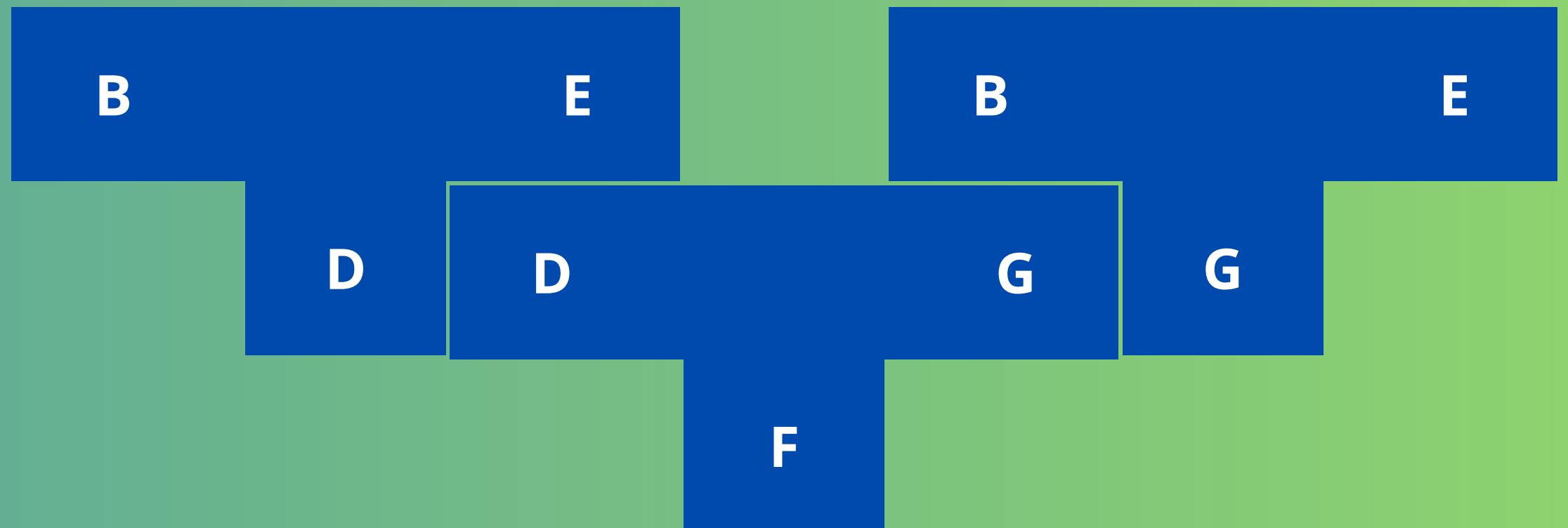


Operaciones con Diagramas T

1)

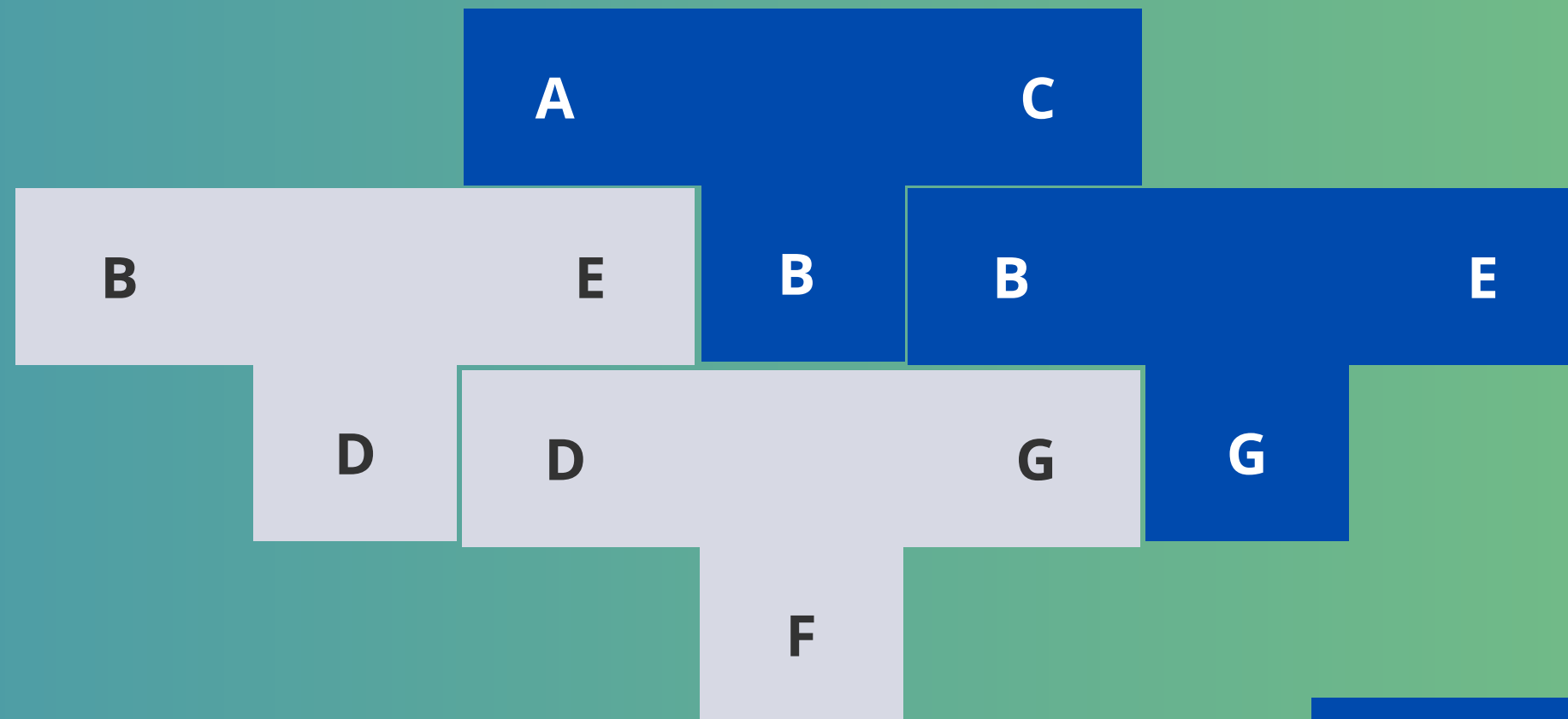


2)



Operaciones con Diagramas T

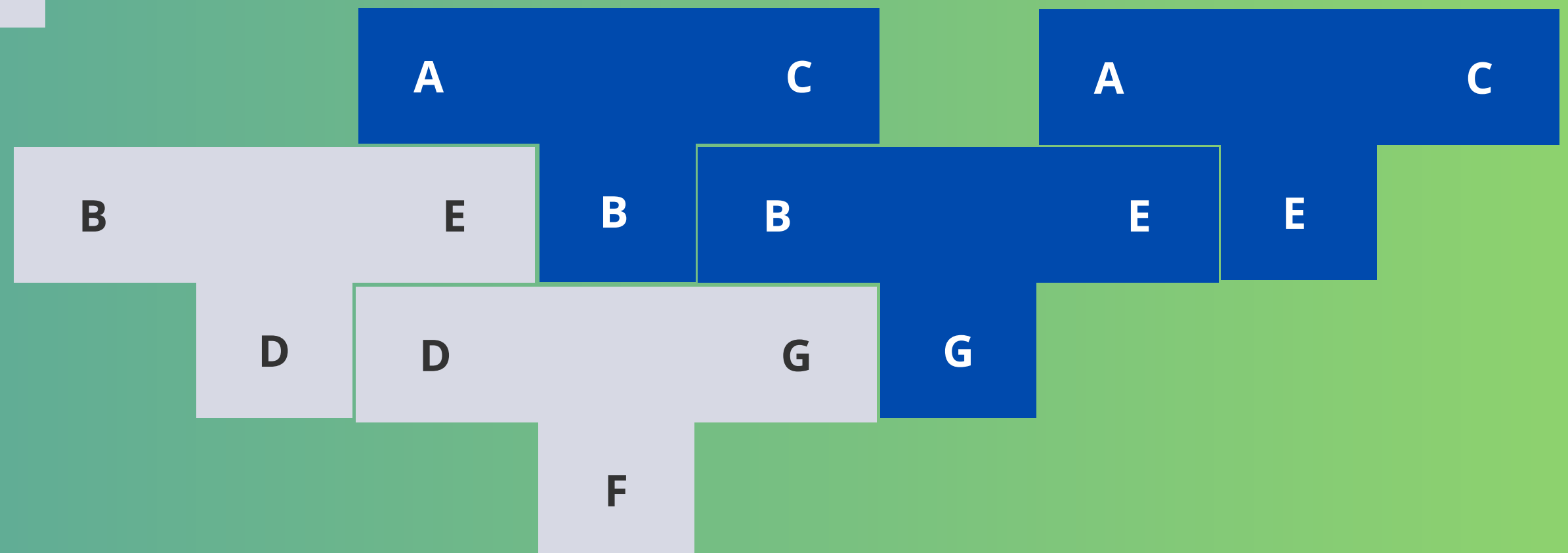
3)



Nota: los que usamos son los azules (los resultantes)

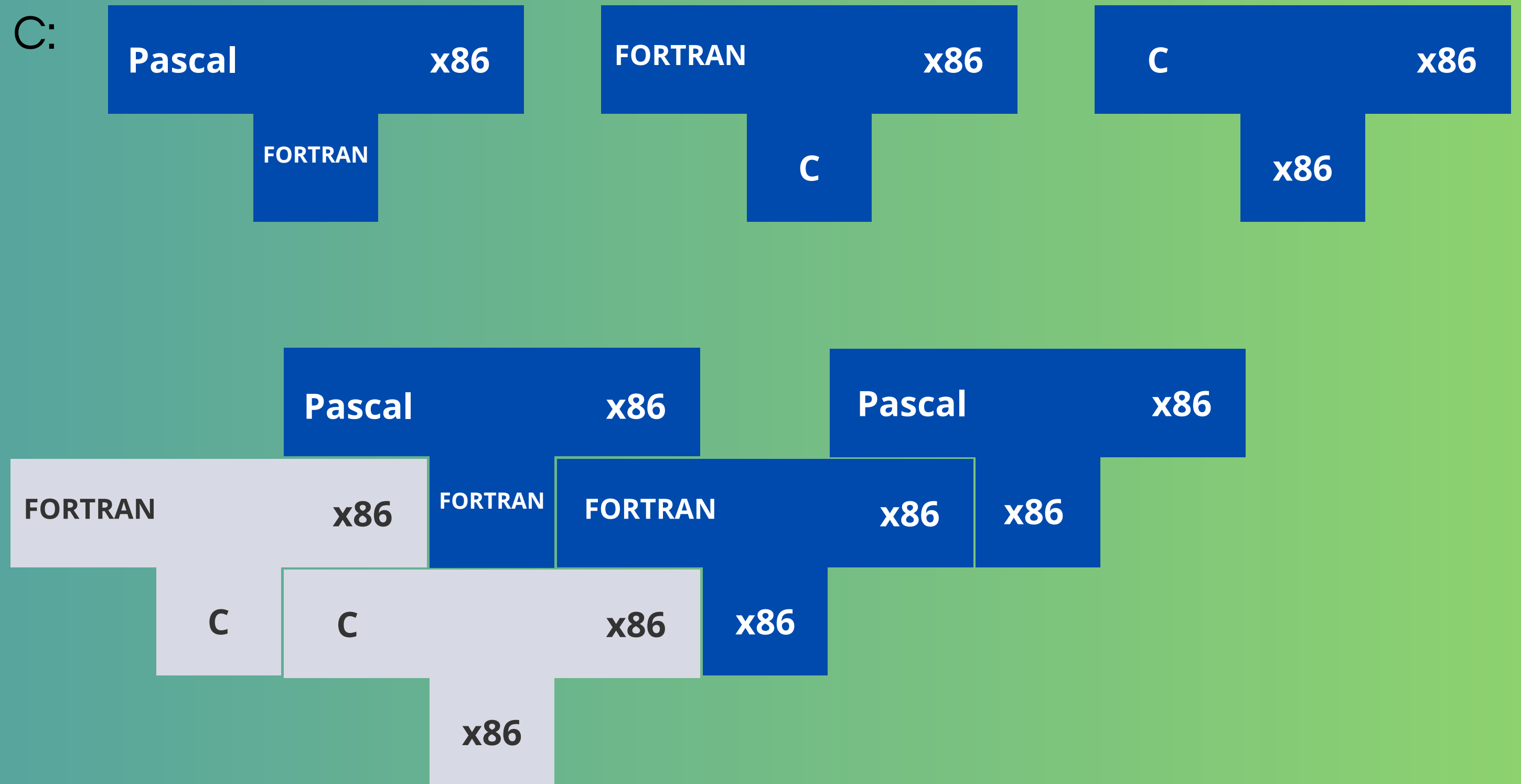


4)



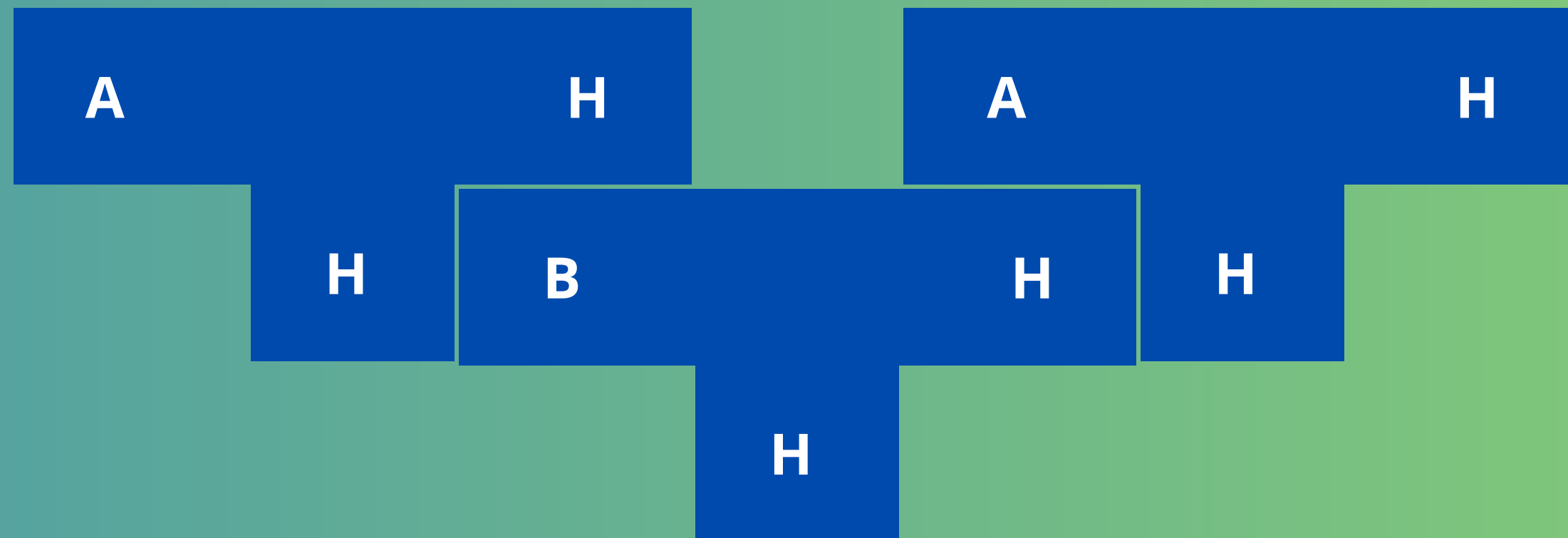
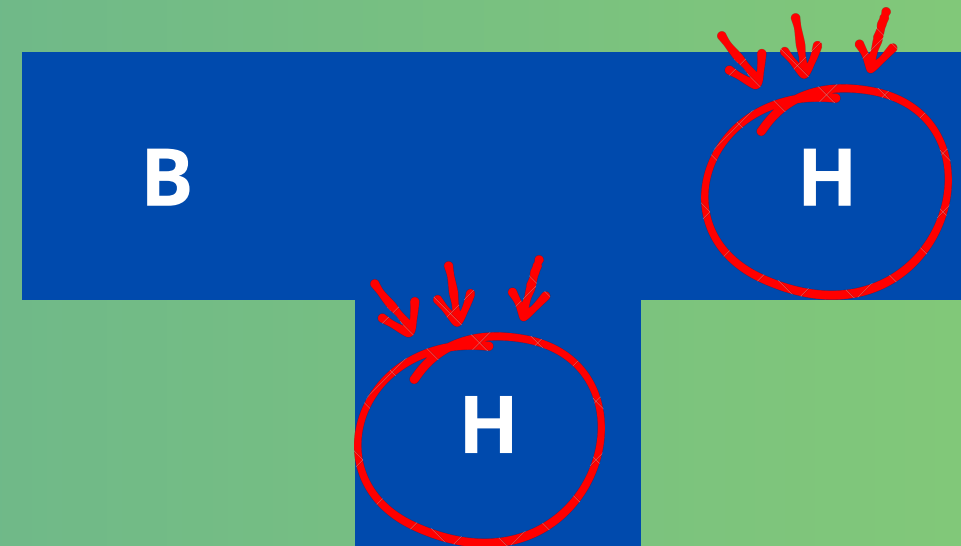
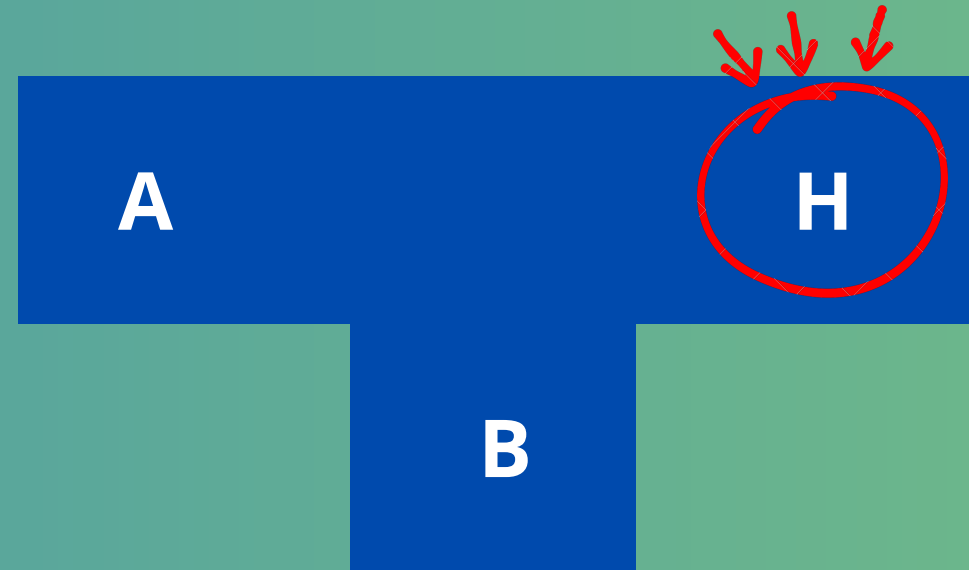
Operaciones con Diagramas T

Pascal - Fortran - C:



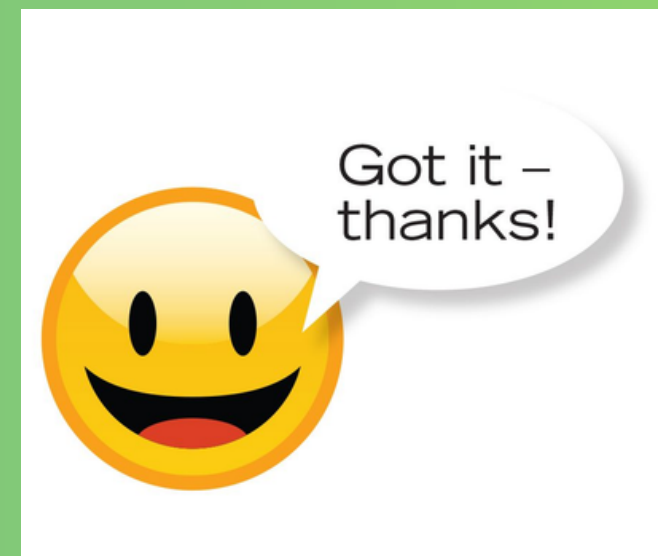
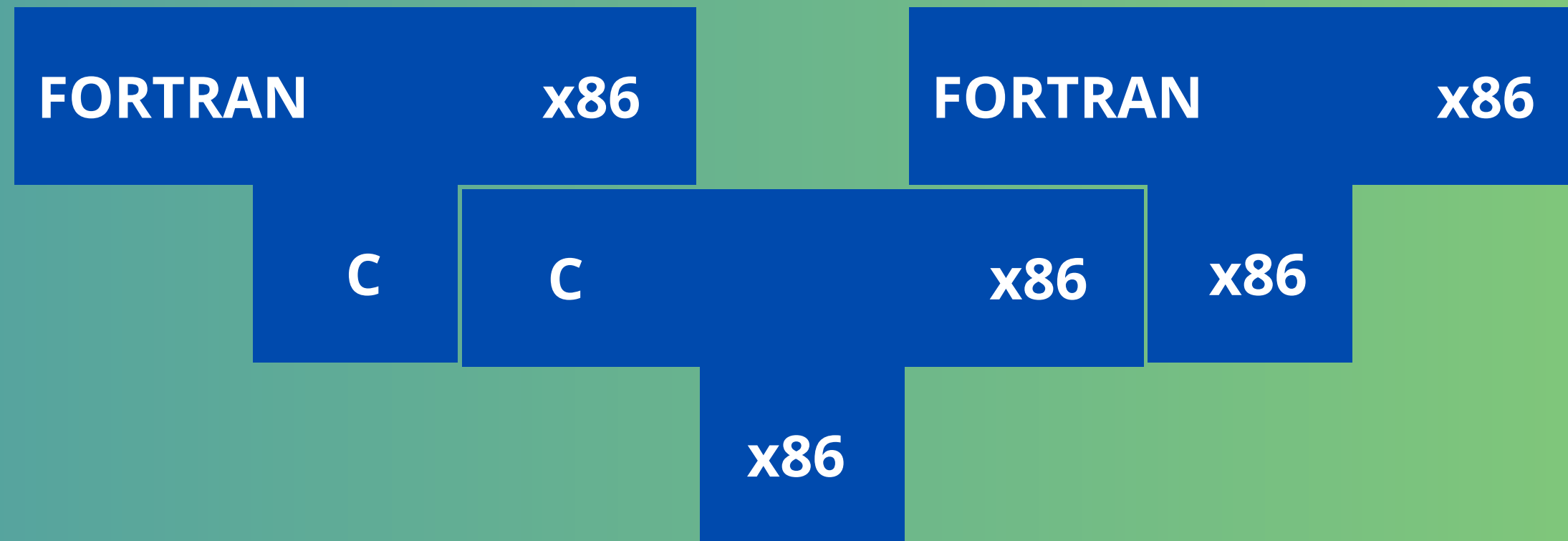
Operaciones con Diagramas T

Caso típico:



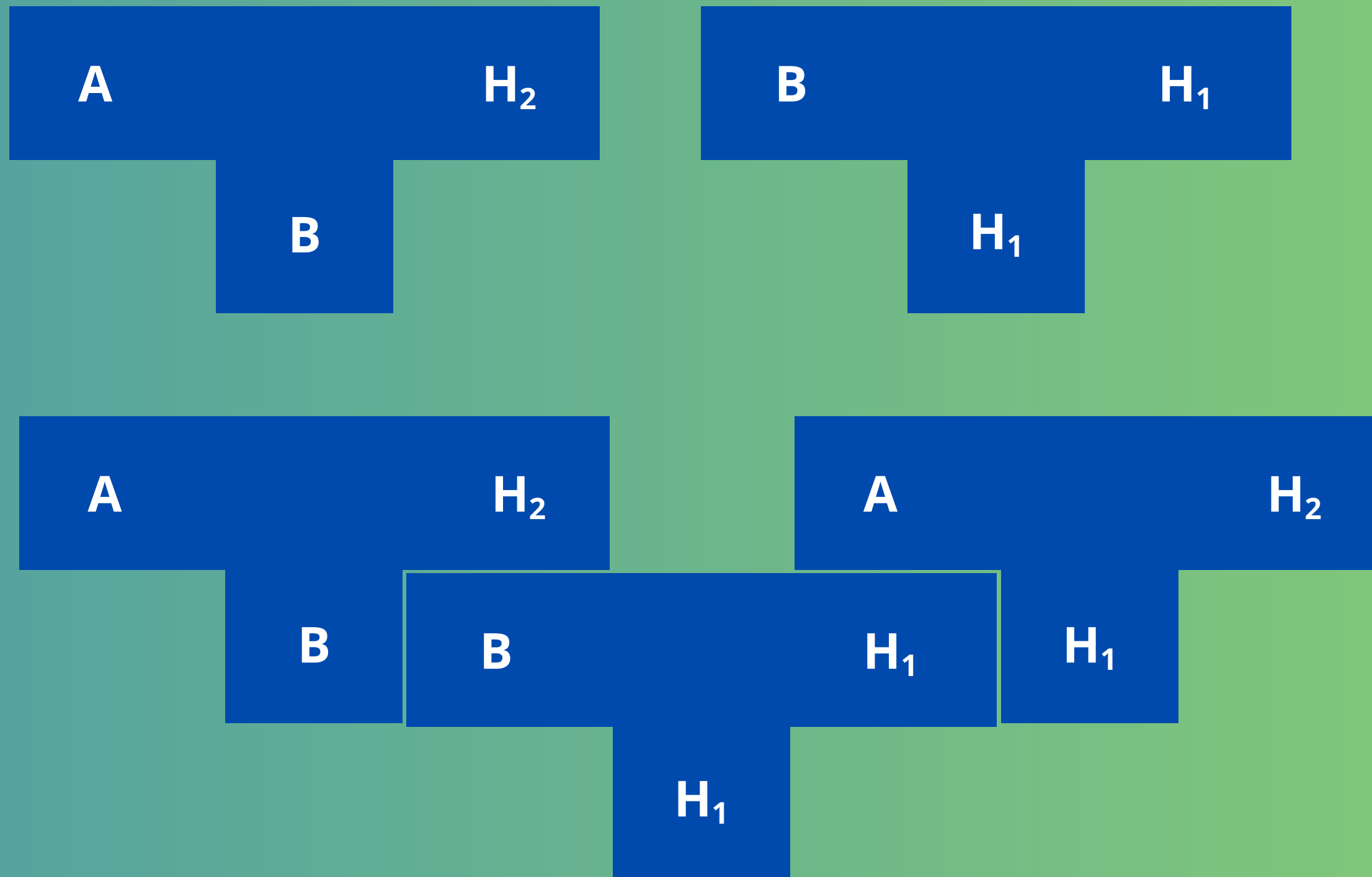
Operaciones con Diagramas T

Ejemplo:



Operaciones con Diagramas T

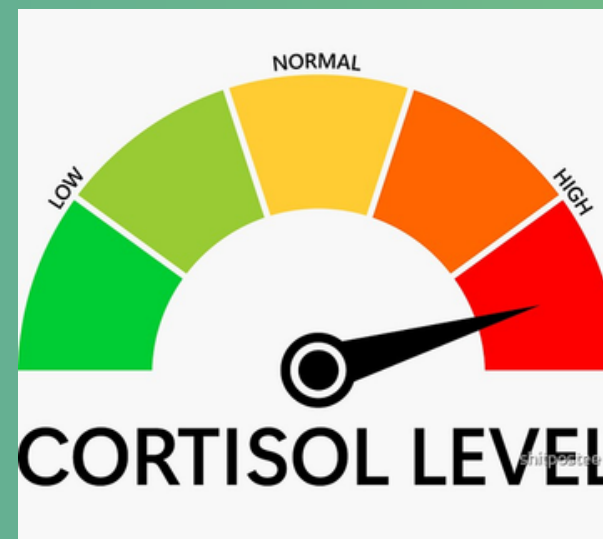
Tercer caso: Lenguaje Objeto es diferente al Lenguaje de Implementación



Operaciones con Diagramas T

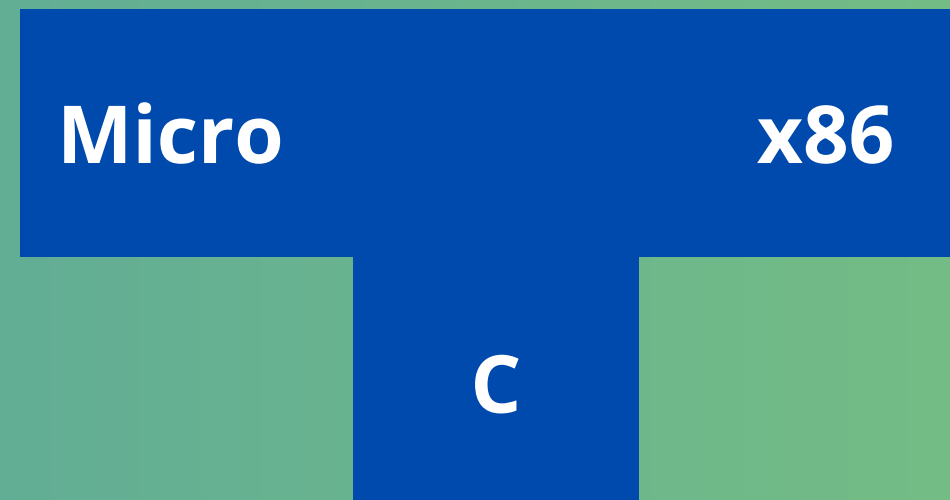
Proyecto 0:

- Recibe Micro
- Escrito en C
- Según el semestre hay diferentes versiones:
 - a. Genera x86
 - b. Genera MIPS (sobre simulador SPIM)
 - c. Genera ARM (sobre simulador en linux)

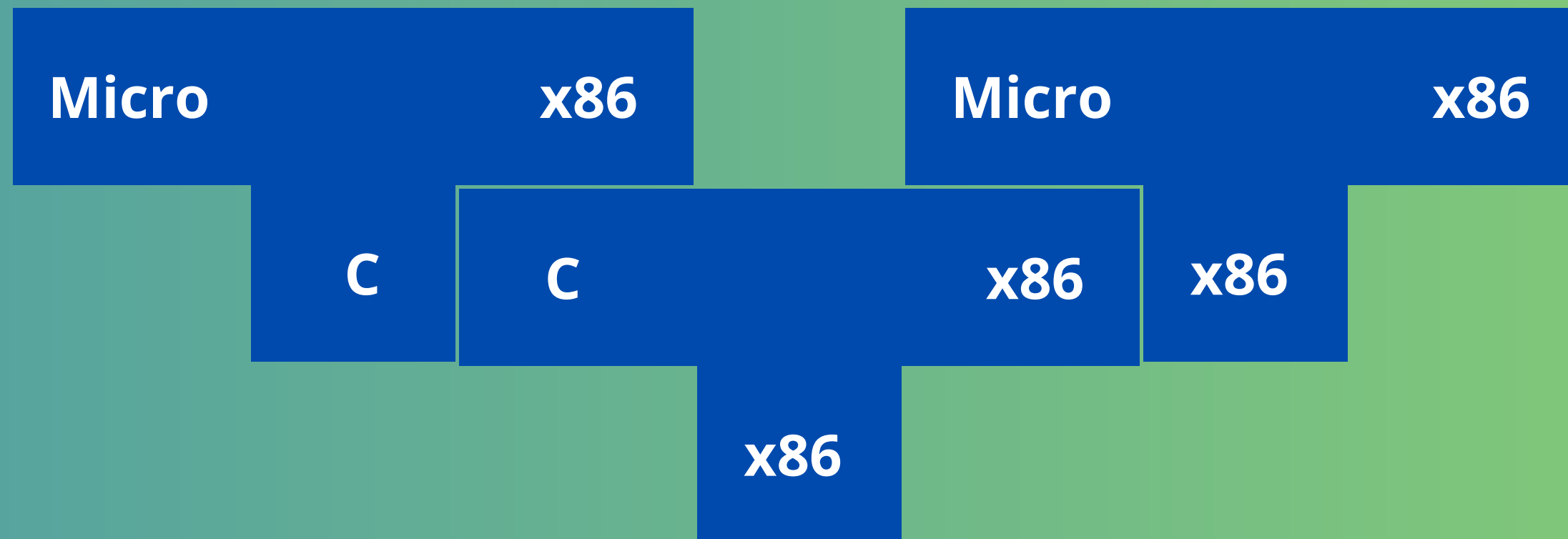


Operaciones con Diagramas T

Variante A:



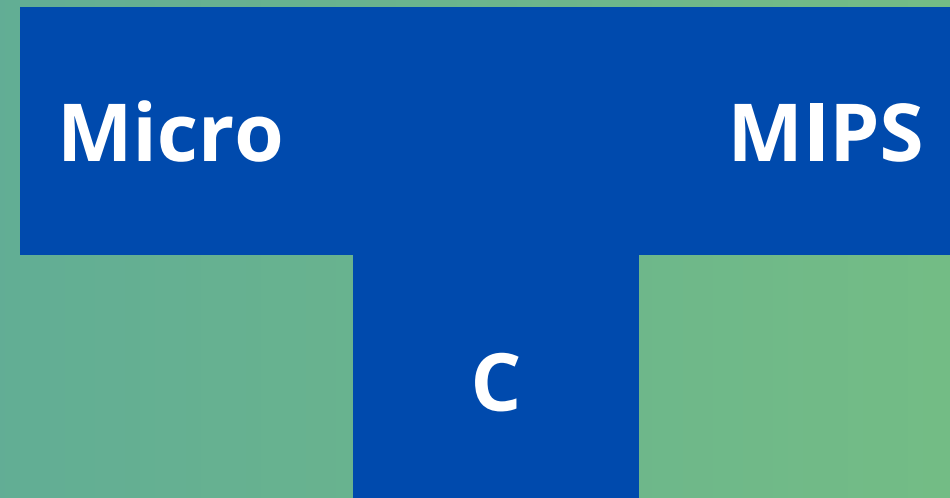
Compilador Normal



Compila en GCC

Operaciones con Diagramas T

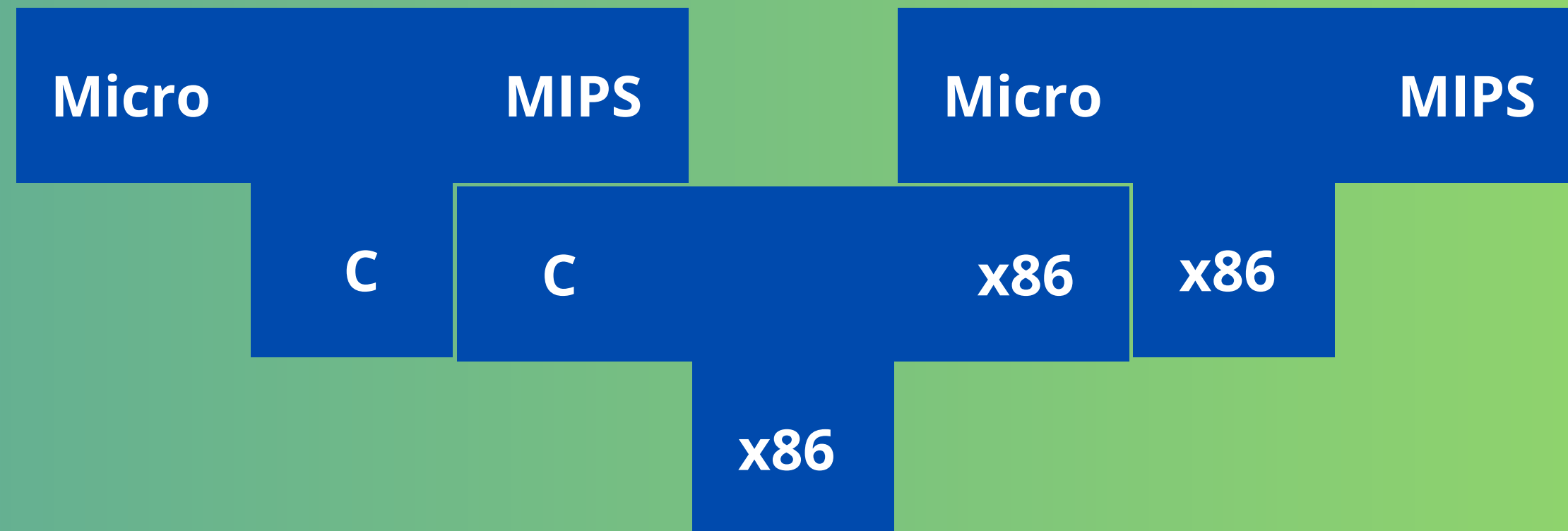
Variante B:



Arquitectura MIPS:

- Es una arquitectura RISC
- Diseño original de 1985
- Tiene varias versiones
- Muy influyente para el diseño RISC estudiado en todo el mundo

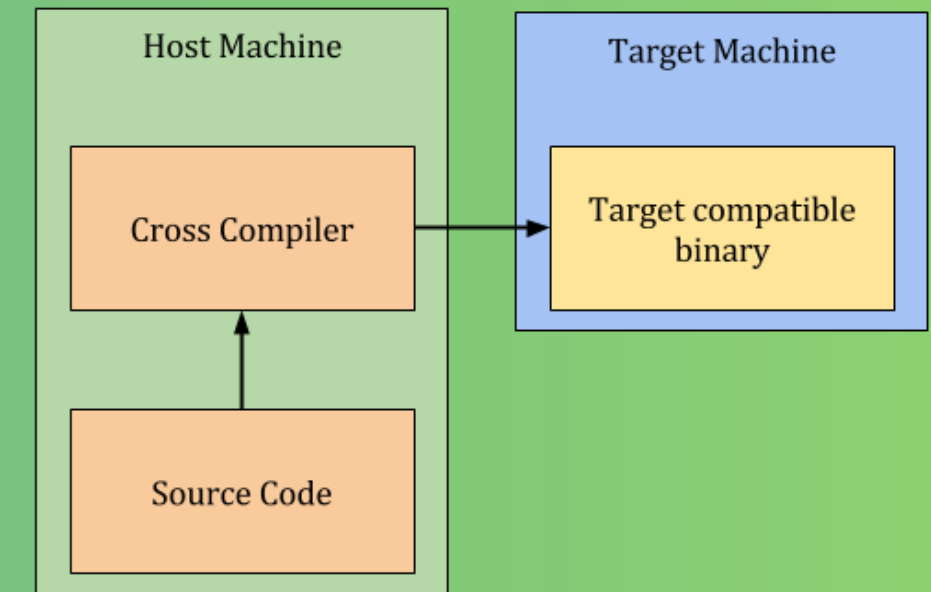
Cross Compiler



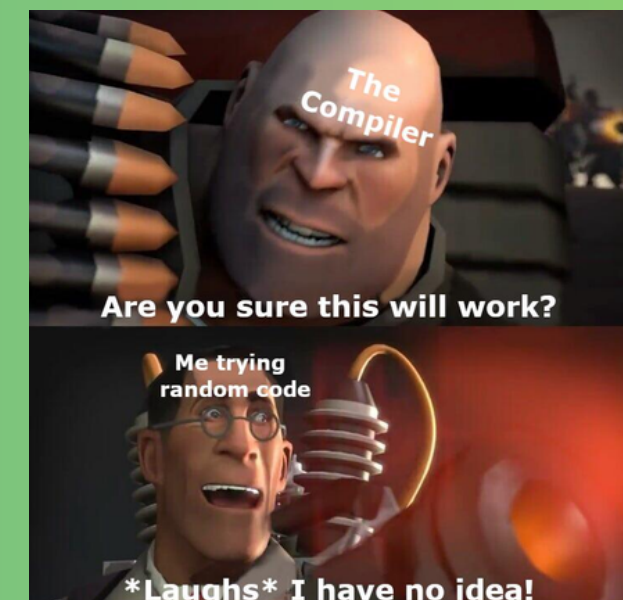
Operaciones con Diagramas T

Compilador Cruzado (Cross Compiler):

- Herramienta de software que corre en un sistema pero genera código ejecutable diseñado para funcionar en un sistema distinto
- Maquina 1 (host) $\neq$ Maquina 2 (target)
- ¿Cuándo se necesita?:
 - Maquina 1 no existe o no la tenemos
 - Maquina 1 tiene limitaciones de recursos para correr un compilador
- Usado en sistemas empuotrados (¿o “embebidos”?)

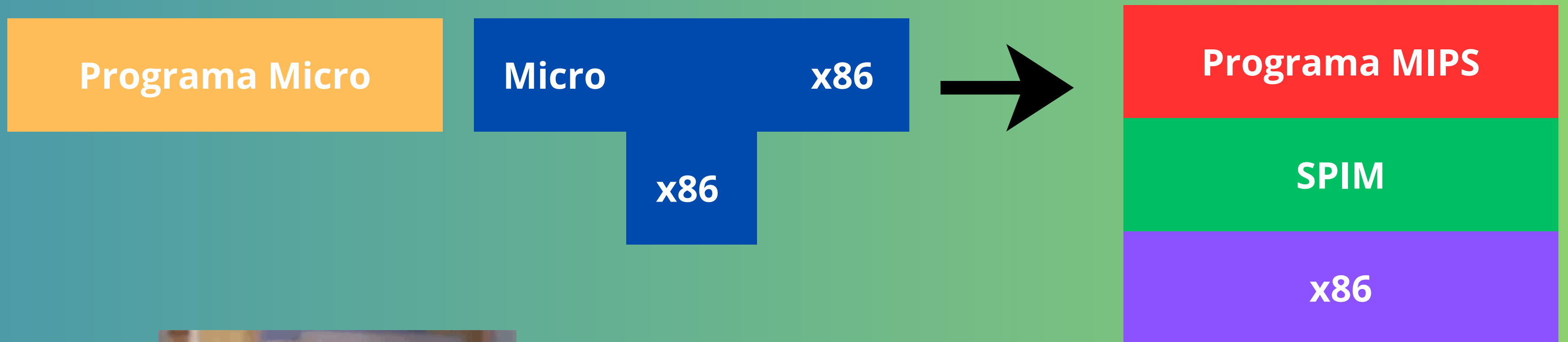


Cross Compiler operation



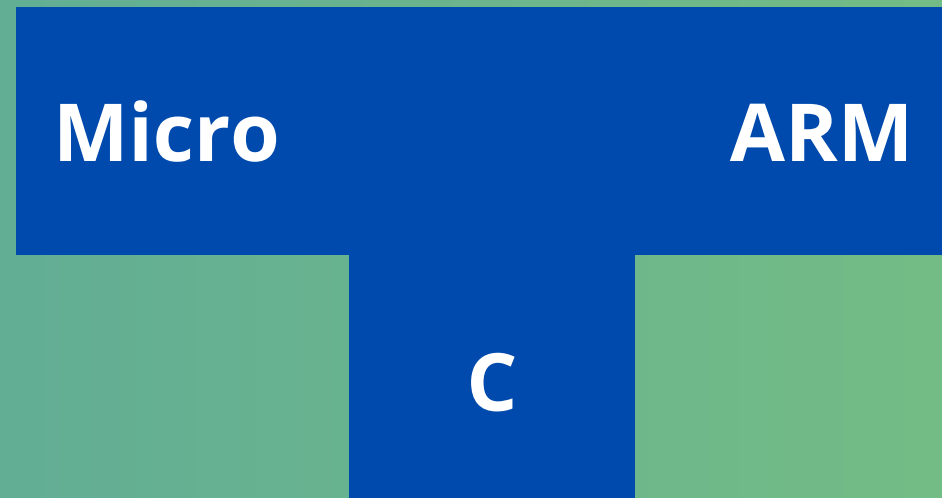
Operaciones con Diagramas T

Variante B:



Operaciones con Diagramas T

Variante C:



Arquitectura ARM:

- Es una arquitectura RISC
- Diseño original de los 80s
- Tiene varias versiones
- Muy eficiente energéticamente
- Usado en celulares, laptops, tablets

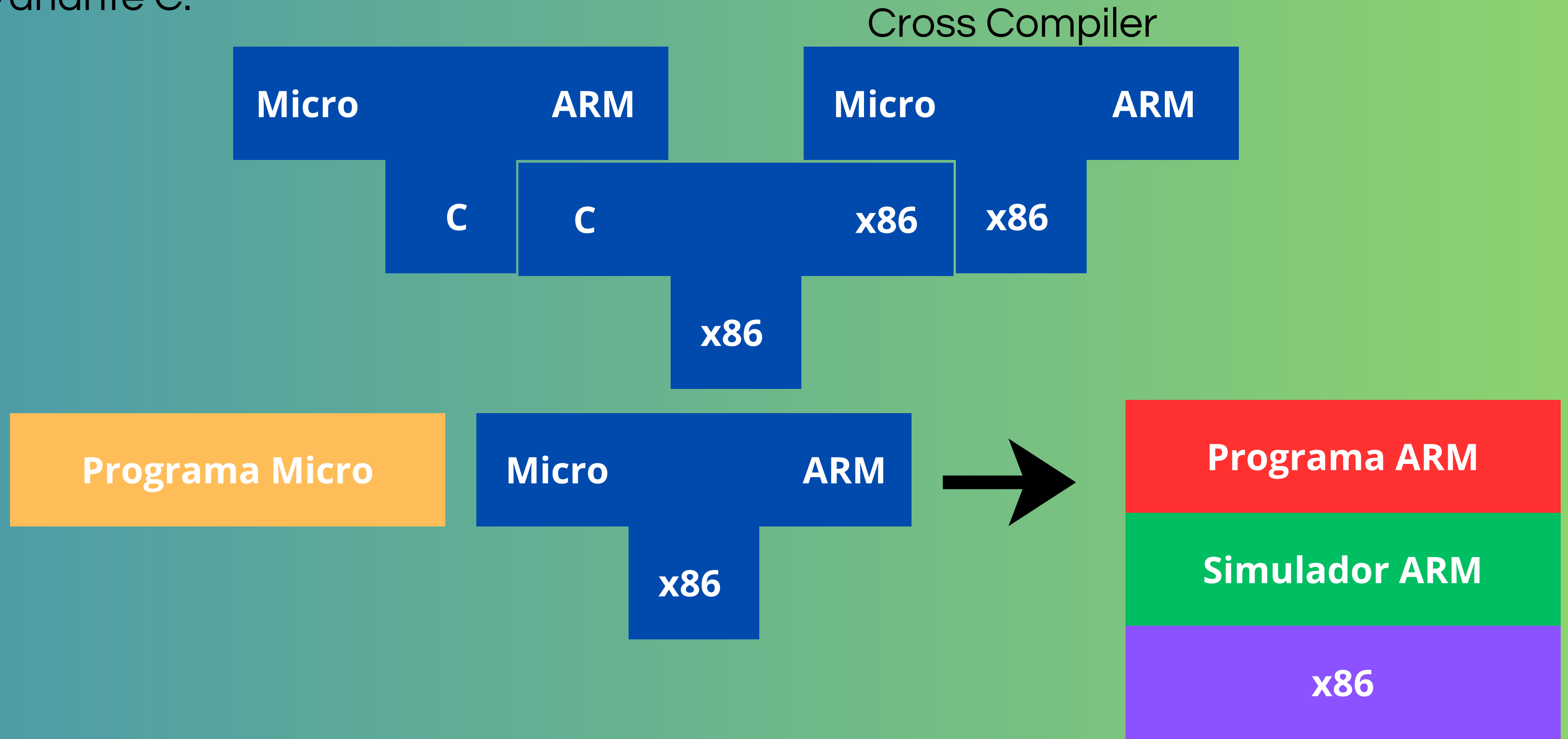
Arquitectura ARM:

- Modelos de 64/32 bits
- Es la arquitectura más usada del mundo (alrededor de 325mil millones al 2026)
- Presente en 99% de todos los smartphones
- Usado en el 30% a 40% de todos los dispositivos electronicos actuales



Operaciones con Diagramas T

Variante C:



Operaciones con Diagramas T

Compilador para ARM en x86:

Cross Compiler

